01 — Product Requirements Document (PRD)

Product: EventPilot — The Living Plan **Challenge:** Google Prompt Wars Virtual — Challenge 5: Celebration & Event Planner **Version:** 1.0 — planning baseline

1. Product summary

EventPilot helps people plan weddings, birthdays and community events using a structured plan that connects budget, timeline, vendor checklist and personalized ideas. When a user changes a supported constraint, the application recalculates what it can deterministically and shows proposed changes for review.

2. Problem statement

A plan generated as a long chat response can be hard to maintain. Budgets, tasks, dates and decisions may become inconsistent after a change. EventPilot aims to make the plan editable, transparent and responsive to changing constraints.

This is a product hypothesis informed by the supplied research, not a claim that every competing product has these limitations.

3. Target users

- A person organizing a birthday or family celebration with a limited budget.
- A couple coordinating a wedding plan and vendor tasks.
- A community organizer managing tasks, deadlines and spending priorities.

4. Goals

- Deliver all four challenge areas: budgets, timelines, vendor checklists, personalized ideas.
- Demonstrate a visible “change one thing → review the impact” workflow.
- Keep arithmetic and schedule rules deterministic and testable.
- Make AI output useful without letting it silently overwrite user decisions.
- Work in a demo environment even when the AI provider is unavailable.

5. Non-goals

No marketplace, booking, payment, contract generation, vendor outreach, RSVP system, or external price verification in MVP. No claim that an AI estimate is a quote.

6. Primary user journey

1. Open app and choose Wedding, Birthday, or Community Event.
2. Enter event date, guest count, total budget, currency, and priorities.
3. Generate a plan or load a sample plan.
4. Review Budget, Timeline, Vendor Checklist and Ideas.
5. Change guest count, date, or budget.
6. Inspect the before/after impact and any assumptions.
7. Accept/reject flexible suggestions; locked decisions remain unchanged.
8. Export or copy a plan summary if feasible within the chosen stack.

7. Functional requirements

- FR-01: User can create an event plan with event type, date, guest count, budget, currency and priorities.
- FR-02: The app presents four planning areas: budget, timeline, vendor checklist and ideas.
- FR-03: Budget totals are calculated by deterministic code, not LLM arithmetic.
- FR-04: Budget values carry provenance: User-entered, AI estimate, or Confirmed by user.
- FR-05: Tasks have title, category, status, due date, optional dependencies, priority and lock state.
- FR-06: Schedule calculations detect invalid dates and dependency cycles.
- FR-07: User can change guest count, event date and total budget.
- FR-08: The app computes a change set and displays a before/after diff.
- FR-09: User can accept/reject proposed flexible changes individually.
- FR-10: User-locked tasks and decisions are not silently modified.
- FR-11: AI suggestions are validated against a schema and rendered as untrusted content.
- FR-12: AI failure produces a helpful message and preserves the editable plan.
- FR-13: A sample/demo plan is available without an API key.
- FR-14: Core interactions work on mobile and desktop and are keyboard accessible.
- FR-15: User can reset the demo or create a new plan without stale state leaking across plans.

8. User stories and acceptance criteria

Setup

- Given valid event details, when the user creates a plan, then the plan view shows all four required planning areas.
- Invalid budget, negative guest count, or invalid date is rejected with an actionable inline message.

Budget

- Given line items, the displayed total equals the sum of included items.
- A budget overrun is visibly identified and never “fixed” by changing values without user approval.
- Monetary calculations use integer minor units where applicable and display formatted currency.

Timeline

- Tasks display due dates and status.
- A dependency cycle is rejected with a clear message.
- Moving the event date recalculates eligible relative dates while preserving locked decisions and highlighting conflicts.

Replanning

- Changing guest count, event date, or budget creates a reviewable change set.
- No proposed change is applied until accepted.
- Rejecting a suggestion leaves the original value intact.

AI

- Valid AI response is parsed and validated before use.
- Invalid or unavailable AI response produces a safe fallback.
- AI content cannot introduce executable HTML or override system/developer instructions.

Accessibility

- All controls have accessible names.
- Core workflow can be completed using keyboard only.
- Focus is visible and errors are associated with relevant fields.

9. MVP prioritization

P0: Event setup, sample plan, budget calculations, four planning sections, responsive UI, basic tests. **P1:** Timeline dependencies, supported constraint changes, reviewable diff, AI ideas with validation. **P2:** Export, advanced scheduler, persistence, richer scenario comparison.

Do not begin P2 until P0 and P1 core journeys work.

10. Success measures (internal targets, not official judging weights)

- All required challenge areas are present in the demo.
- 100% pass rate for required deterministic unit tests.
- No known blocking errors in production build.
- No known application-console errors in tested core journeys.
- No known failed essential requests in the tested environment.
- No unresolved critical security issues.
- Keyboard accessibility and responsive smoke tests pass.

11. Assumptions and open questions

- Challenge deadline and exact rules must be verified.
- Currency defaults should be user-selectable; do not hard-code a market price.
- Persistence defaults to local/browser state if acceptable and adequate.
- Gemini is optional for core functionality; provider availability must not block the demo.